Arrhythmia Model Component Information

1. Preprocessing and Feature Extraction Code

This code handles filtering, R-peak detection, beat segmentation, and morphological feature extraction (from backend/AI_models/arrhythmia/inference.py).

```
  def _lead_features(signal: np.ndarray, sampling_rate: float) -> dict[str, float] | None:
      if signal.size < int(sampling_rate * 2.5) or np.ptp(signal) < 1e-6:
          return None
      # Filtering / Normalization
      normalized = (signal - np.median(signal)) / (np.std(signal) + 1e-8)

      # R-peak detection
      distance = max(1, int(0.25 * sampling_rate))
      peaks, _ = find_peaks(normalized, distance=distance, prominence=0.45)
      if len(peaks) < 3:
          return None

      # Beat segmentation
      r = int(peaks[-2])
      pre_rr = float(peaks[-2] - peaks[-3])
      post_rr = float(peaks[-1] - peaks[-2])
      radius = max(2, int(0.18 * sampling_rate))
      left, right = max(0, r - radius), min(len(normalized), r + radius + 1)
      beat = normalized[left:right]

      # Feature Extraction (Morphological & Intervals)
      q_rel = int(np.argmin(beat[: max(1, r - left + 1)]))
      s_start = min(len(beat) - 1, r - left)
      s_rel = s_start + int(np.argmin(beat[s_start:]))
      p_area = normalized[max(0, r - int(.25 * sampling_rate)): max(1, r - int(.08 * sampling_rate))]
      t_area = normalized[min(len(normalized), r + int(.08 * sampling_rate)): min(len(normalized), r + int(.40
      p_peak = float(np.max(p_area)) if p_area.size else 0.0
      t_peak = float(np.max(t_area)) if t_area.size else 0.0
      q_index, s_index = left + q_rel, left + s_rel
      morphology_positions = np.linspace(left, right - 1, 5).astype(int)

      return {
          "pre-RR": pre_rr,
          "post-RR": post_rr,
          "pPeak": p_peak,
          "tPeak": t_peak,
          "rPeak": float(normalized[r]),
          "sPeak": float(normalized[s_index]),
          "qPeak": float(normalized[q_index]),
          "qrs_interval": float(max(1, s_index - q_index)),
          "pq_interval": float(max(1, q_index - (r - int(.18 * sampling_rate)))),
          "qt_interval": float(max(1, (r + int(.36 * sampling_rate)) - q_index)),
          "st_interval": float(max(1, int(.08 * sampling_rate))),
          **{f"qrs_morph{i}": float(normalized[position]) for i, position in enumerate(morphology_positions)},
      }
```

2. Hub Arrhythmia Logic Code

This code handles Bradycardia, Tachycardia, Belt posture data correlation, and the Master Risk Score (from backend/main.py and backend/AI_models/sensor_fusion.py).

```
# backend/main.py: Bradycardia and Tachycardia detection
def evaluate_rules(self, vitals, device_id):
    hr = vitals.get("heart_rate")
    if hr is not None:
        if 0 < hr < 45:
            # Bradycardia detection (with debounce logic)
            self.bradycardia_counts[device_id] += 1
            if self.bradycardia_counts[device_id] >= 10:
                alerts.append({"diagnosis": "SUSTAINED BRADYCARDIA", "severity": 2, "trigger": "LOW_HR"})
        else:
            self.bradycardia_counts[device_id] = 0
            if hr > 140:
                # Tachycardia detection
                alerts.append({"diagnosis": "TACHYCARDIA", "severity": 2, "trigger": "HIGH_HR"})

# backend/AI_models/sensor_fusion.py: Posture Data Usage
def evaluate(self, device_id, wrist_vitals, history):
    # Cross-verifies wrist vitals with waist motion/posture
    waist = self._waist_state[device_id]
    activity = waist["posture"] # e.g. RUNNING, LYING
    expected = self.ACTIVITY_EXPECTED.get(activity, ...)
    hr = wrist_vitals.get("heart_rate")

    # Determines if tachycardia is explained by physical activity
    hr_explanation = self._explain_vital("heart_rate", hr, expected["hr_range"], activity)
    should_suppress_hr = (hr_explanation == "activity_explained" and fusion_confidence > 0.5)

# backend/main.py: Master Risk Score Integration
# Suppress alerts based on posture/fusion
if suppress_hr_fusion:
    rules_alerts = [a for a in rules_alerts if a.get("trigger") not in ("HIGH_HR",)]

# Arrhythmia Model severity integration
arr_screening = screenings.get("arrhythmia", {})
if arr_screening.get("status") == "ready" and arr_screening.get("severity", 0) >= 2:
    rules_alerts.insert(0, {
        "diagnosis": f"ARRHYTHMIA: {arr_screening.get('diagnosis')}",
        "severity": arr_screening.get("severity", 2),
    })

# Master Risk Score Calculation
risk_score = 100 if is_emergency else 0
if not is_emergency:
    ready_likelihoods = [s["likelihood_percent"] for s in screenings.values()]
    correlation_score = correlation_result.get("correlation_score", 0)
    if ready_likelihoods:
        model_max = max(ready_likelihoods)
        # Blend: 70% model max likelihood + 30% correlation score
        risk_score = int(round(0.7 * model_max + 0.3 * correlation_score))
```